inmind.academy
by inmind.ai

State management and optimization

SPRING 2026

Overview

- State management
- Optimization techniques

State management

The component spaghetti problem

- As apps grow, components need to share data (user info, shopping cart, filters, etc.)
- Without structure, you end up passing data through multiple component layers (@Input/@Output chains)
- Sibling components can't easily communicate
- Same data gets duplicated across components, causing sync issues
- Hard to track where state changes happen

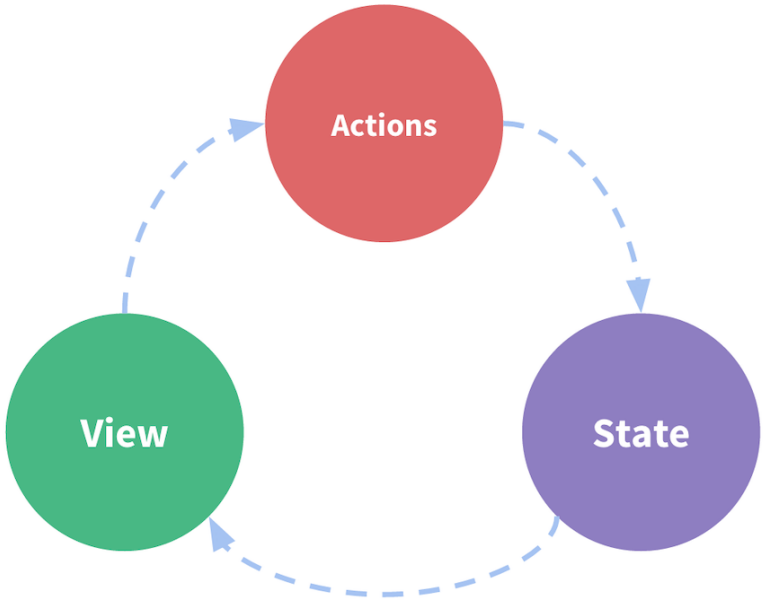
Redux

Three Principles:

1.Single Source of Truth: All application state lives in one place (the store)

2.State is Read-Only and Immutable: You can't mutate state directly; you dispatch actions to describe what happened

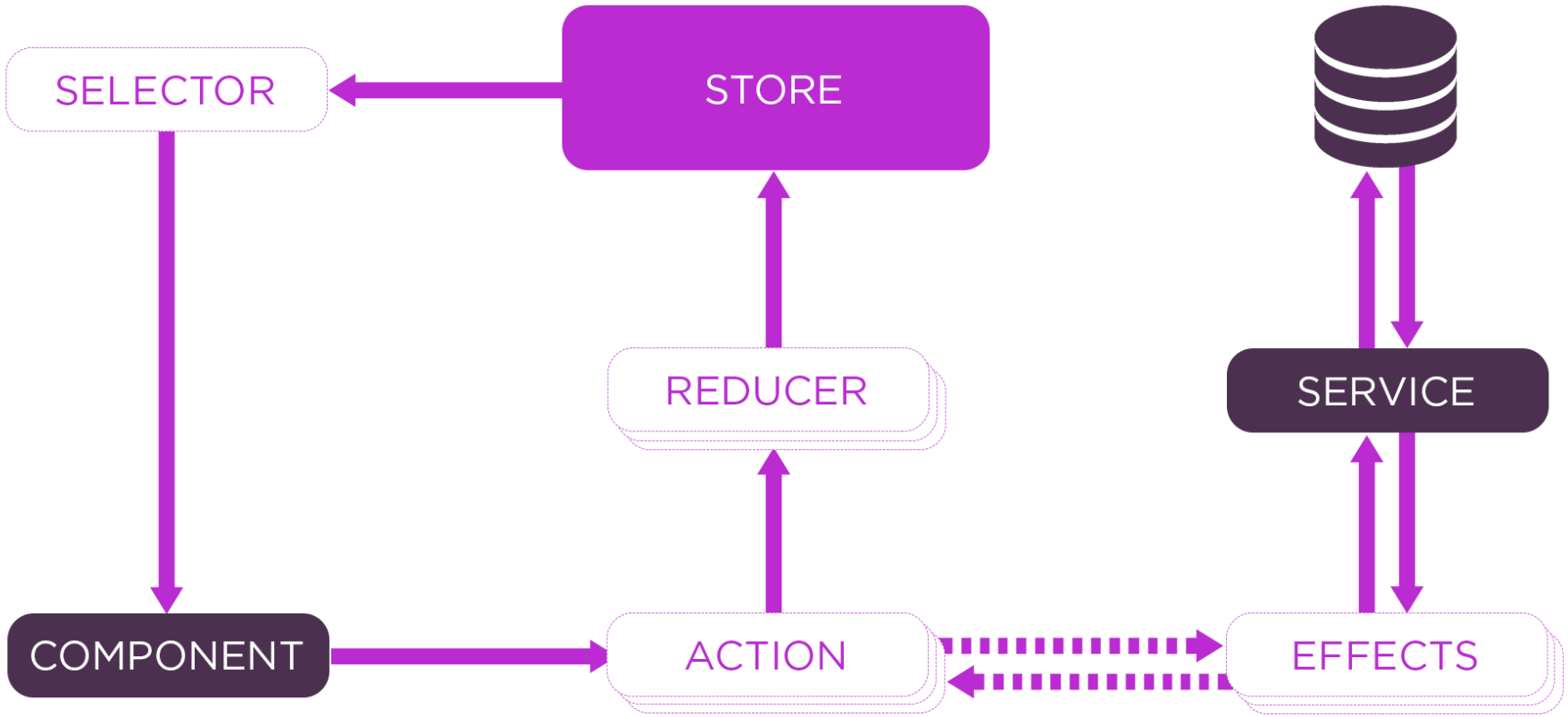
3.Changes via Pure Functions and Unidirectional flow: Reducers (pure functions) specify how state transforms in response to actions to generate new state.



Core Building Blocks:

inmind.academy
by inmind.ai

- **Store**: Single state tree for your entire app
- **Actions**: Objects describing events ({ type: '[Cart] Add Item', payload: item })
- **Reducers**: Pure functions that take current state + action → new state
- **Selectors**: Functions to query/derive data from the store
- **Effects**: Handle side effects (API calls, routing, etc.)



Install NgRx in project

Run the following command in your terminal to install NgRx to your project.

```
ng add @ngrx/store@latest
```

Define State Interface

```ts
// store/cart/cart.state.ts
export interface CartState {
  items: CartItem[];
  total: number;
  loading: boolean;
  error: string | null;  // Add error tracking
}

export const initialState: CartState = {
  items: [],
  total: 0,
  loading: false,
  error: null
};
```

```ts
export interface CartItem {
  id: number;
  name: string;
  price: number;
  quantity: number;
}
```

Define the variables in your state that should be tracked of and reused.

Create Actions


```ts
// store/cart/cart.actions.ts
import { createAction, props } from '@ngrx/store';
import { CartItem } from './cart.state';

// Load Cart Actions
export const loadCart = createAction('[Cart] Load Cart');

export const loadCartSuccess = createAction(
  '[Cart] Load Cart Success',
  props<{ items: CartItem[] }>()
);

export const loadCartFailure = createAction(
  '[Cart] Load Cart Failure',
  props<{ error: string }>()
);

// Add Item Actions
export const addToCart = createAction(
  '[Cart] Add Item',
  props<{ product: CartItem }>()
);

export const addToCartSuccess = createAction(
  '[Cart] Add Item Success',
  props<{ product: CartItem }>()
);

export const addToCartFailure = createAction(
  '[Cart] Add Item Failure',
  props<{ error: string }>()
);
```

Defines what kind of actions will happen to the state and what is needed for these actions (props), without the actual logic.

The logic that creates new state is handled by reducers. Next slide!

Create Reducer


```ts
// store/cart/cart.reducer.ts
import { createReducer, on } from '@ngrx/store';
import { initialState } from './cart.state';
import * as CartActions from './cart.actions';

export const cartReducer = createReducer(
  initialState,

  // Loading cart
  on(CartActions.loadCart, (state) => ({
    ...state,
    loading: true,
    error: null
  })),

  on(CartActions.loadCartSuccess, (state, { items }) => ({
    ...state,
    items,
    total: items.reduce((sum, item) => sum + (item.price * item.quantity), 0),
    loading: false,
    error: null
  })),

  on(CartActions.loadCartFailure, (state, { error }) => ({
    ...state,
    loading: false,
    error
  })),

  // Adding item
  on(CartActions.addToCart, (state) => ({
    ...state,
    loading: true
  })),

  on(CartActions.addToCartSuccess, (state, { product }) => {
    const existingItem = state.items.find(item => item.id === product.id);

    if (existingItem) {
      const updatedItems = state.items.map(item =>
        item.id === product.id
          ? { ...item, quantity: item.quantity + 1 }
          : item
      );
      return {
        ...state,
        items: updatedItems,
        total: state.total + product.price,
        loading: false,
        error: null
      };
    }

    return {
      ...state,
      items: [...state.items, { ...product, quantity: 1 }],
      total: state.total + product.price,
      loading: false,
      error: null
    };
  }),

  on(CartActions.addToCartFailure, (state, { error }) => ({
    ...state,
    loading: false,
    error
  }))
);
```

Selectors

```
// store/cart/cart.selectors.ts
import { createFeatureSelector, createSelector } from '@ngrx/store';
import { CartState } from './cart.state';

export const selectCartState = createFeatureSelector<CartState>('cart');

export const selectCartItems = createSelector(
  selectCartState,
  (state) => state.items
);

export const selectCartTotal = createSelector(
  selectCartState,
  (state) => state.total
);

export const selectCartLoading = createSelector(
  selectCartState,
  (state) => state.loading
);

export const selectCartError = createSelector(
  selectCartState,
  (state) => state.error
);
```

Benefits: Memoization - selectors only recalculate when input changes (you can see the similarity with computed signals)

Effects

```
// store/cart/cart.effects.ts
import { Injectable, inject } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { of } from 'rxjs';
import { map, mergeMap, catchError } from 'rxjs/operators';

import { CartService } from '../../services/cart.service';
import * as CartActions from './cart.actions';

@Injectable()
export class CartEffects {
  private actions$ = inject(Actions);
  private cartService = inject(CartService);
```

```
// Effect 1: Load cart from API
loadCart$ = createEffect(() =>
  this.actions$.pipe(
    // Listen for this action
    ofType(CartActions.loadCart),

    // Call the service
    mergeMap(() =>
      this.cartService.getCart().pipe(
        // On success, dispatch success action
        map(items => CartActions.loadCartSuccess({ items })),

        // On error, dispatch failure action
        catchError(error =>
          of(CartActions.loadCartFailure({ error: error.message }))
        )
      )
    )
  )
);
```

Register in app.config

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),

    // HTTP Client (needed for effects that make API calls)
    provideHttpClient(),

    // NgRx Store
    provideStore({
      cart: cartReducer
    }),

    // NgRx Effects - Register here!
    provideEffects([CartEffects]),  // ← Add this line!

    // DevTools
    provideStoreDevtools({
      maxAge: 25,
      logOnly: false
    })
  ]
};
```

Use in Component


```
export class CartComponent implements OnInit {
  private store = inject(Store);

  items$ = this.store.select(CartSelectors.selectCartItems);
  total$ = this.store.select(CartSelectors.selectCartTotal);
  loading$ = this.store.select(CartSelectors.selectCartLoading);
  error$ = this.store.select(CartSelectors.selectCartError);

  ngOnInit() {
    // Dispatch action - effect will handle API call
    this.store.dispatch(CartActions.loadCart());
  }

  addSampleProduct() {
    const product: CartItem = {
      id: Date.now(),
      name: 'Sample Product',
      price: 99,
      quantity: 1
    };

    // Dispatch action - effect will handle API call
    this.store.dispatch(CartActions.addToCart({ product }));
  }
}
```

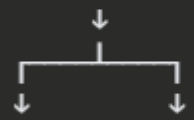
```
@Component({
  selector: 'app-cart',
  standalone: true,
  imports: [CommonModule],
  template: `
    <div class="cart">
      <h2>Shopping Cart</h2>

      <!-- Loading State -->
      <div *ngIf="loading$ | async">Loading...</div>

      <!-- Error State -->
      <div *ngIf="error$ | async as error" class="error">
        Error: {{ error }}
      </div>

      <!-- Cart Items -->
      <div *ngFor="let item of items$ | async" class="item">
        {{ item.name }} - ${{ item.price }} x {{ item.quantity }}
      </div>

      <p>Total: ${{ total$ | async }}</p>

      <!-- Demo: Add a product -->
      <button (click)="addSampleProduct()">
        Add Sample Product
      </button>
    </div>
  `
```

LoadUsers flow explanation


```
// Component dispatches ONE action
this.store.dispatch(CartActions.loadCart());
```

What Happens:

1. Action dispatched: loadCart()



2. BOTH run simultaneously:

REDUCER:
- Receives loadCart()
- Sets loading: true
- Returns new state

EFFECT:
- Listens for loadCart()
- Calls cartService.getCart()
- Waits for response

↓

3. API responds

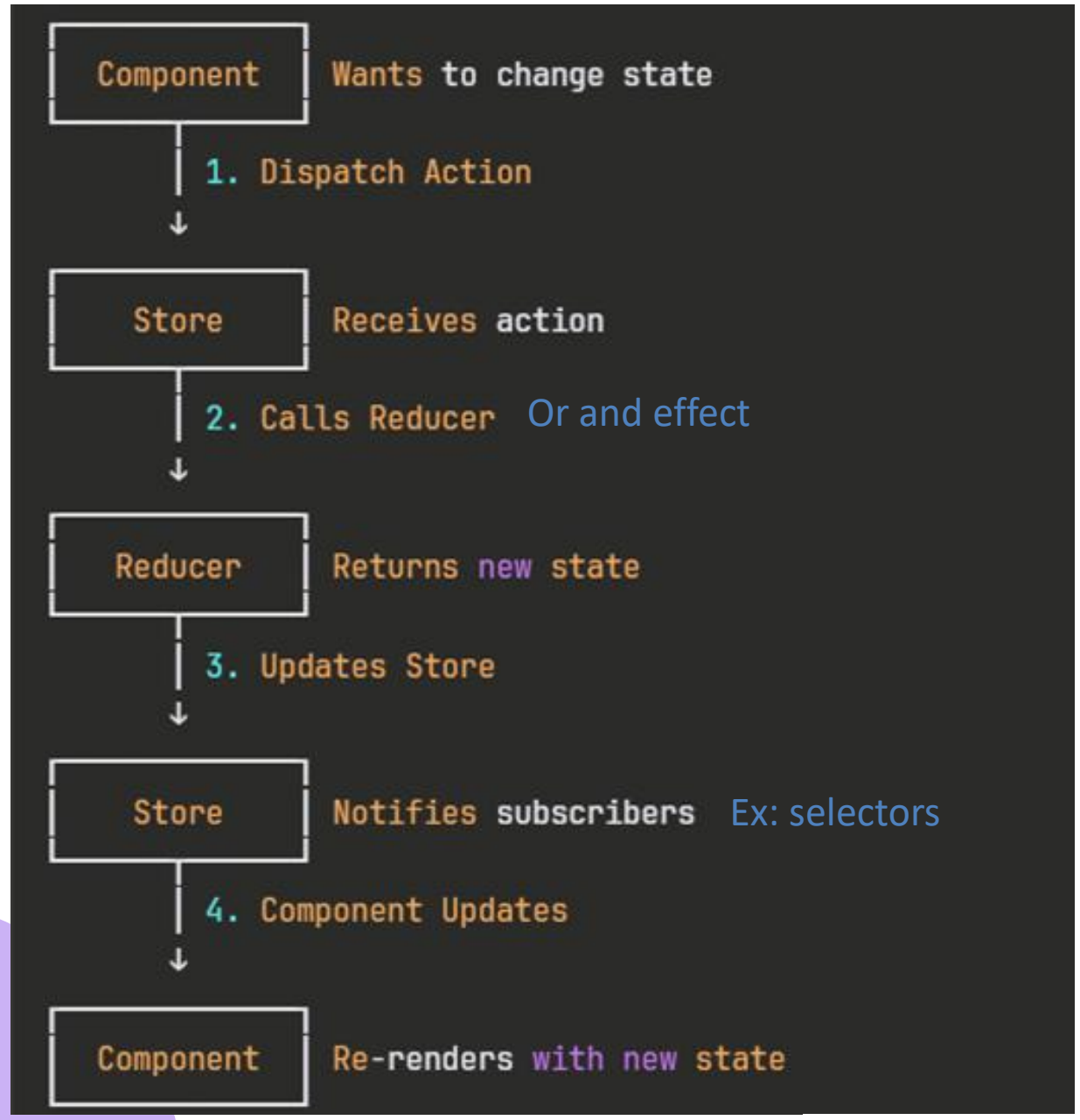
↓

4. Effect dispatches NEW action:
   - loadCartSuccess({ items })
     OR
   - loadCartFailure({ error })

↓

5. REDUCER handles success/failure:
   - Sets loading: false
   - Updates items (success)
     OR
   - Sets error (failure)

Full flow recapitulation



Signal comparison


```
// Everything in ONE file
// Easy to understand at a glance
export class CartService {
  private items = signal<CartItem[]>([]);

  addToCart(item: CartItem) {
    this.items.update(items => [...items, item]);
  }
}
```

- Easy to learn
- All in one place
- Modern
- Less boilerplate
- Innate to Angular (not an external library)

Then, why?

- Standardized approach
- DevTools

Devtools

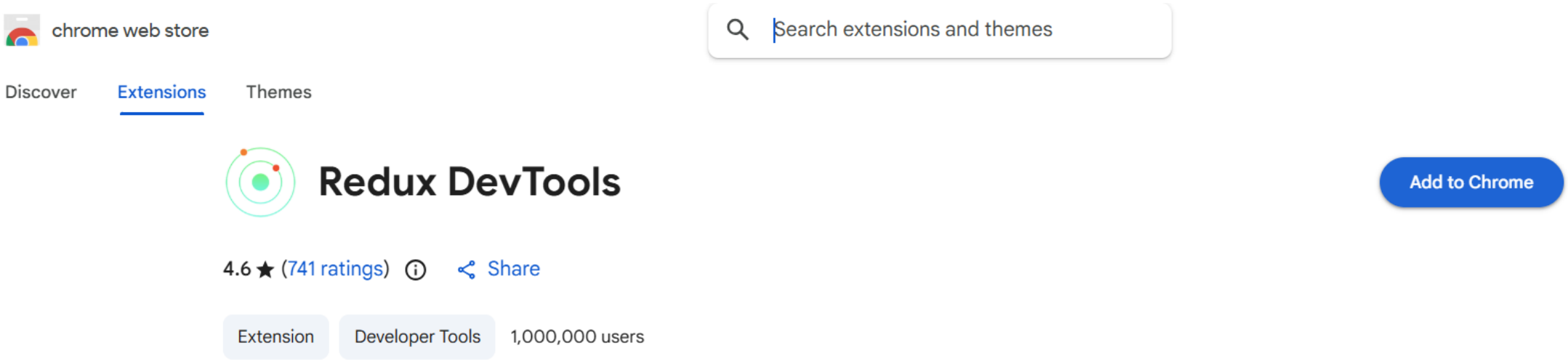
NgRx DevTools are a debugging and inspection toolset for apps using NgRx Store (Redux-style state management for Angular).

They integrate with the Redux DevTools browser extension and let you:

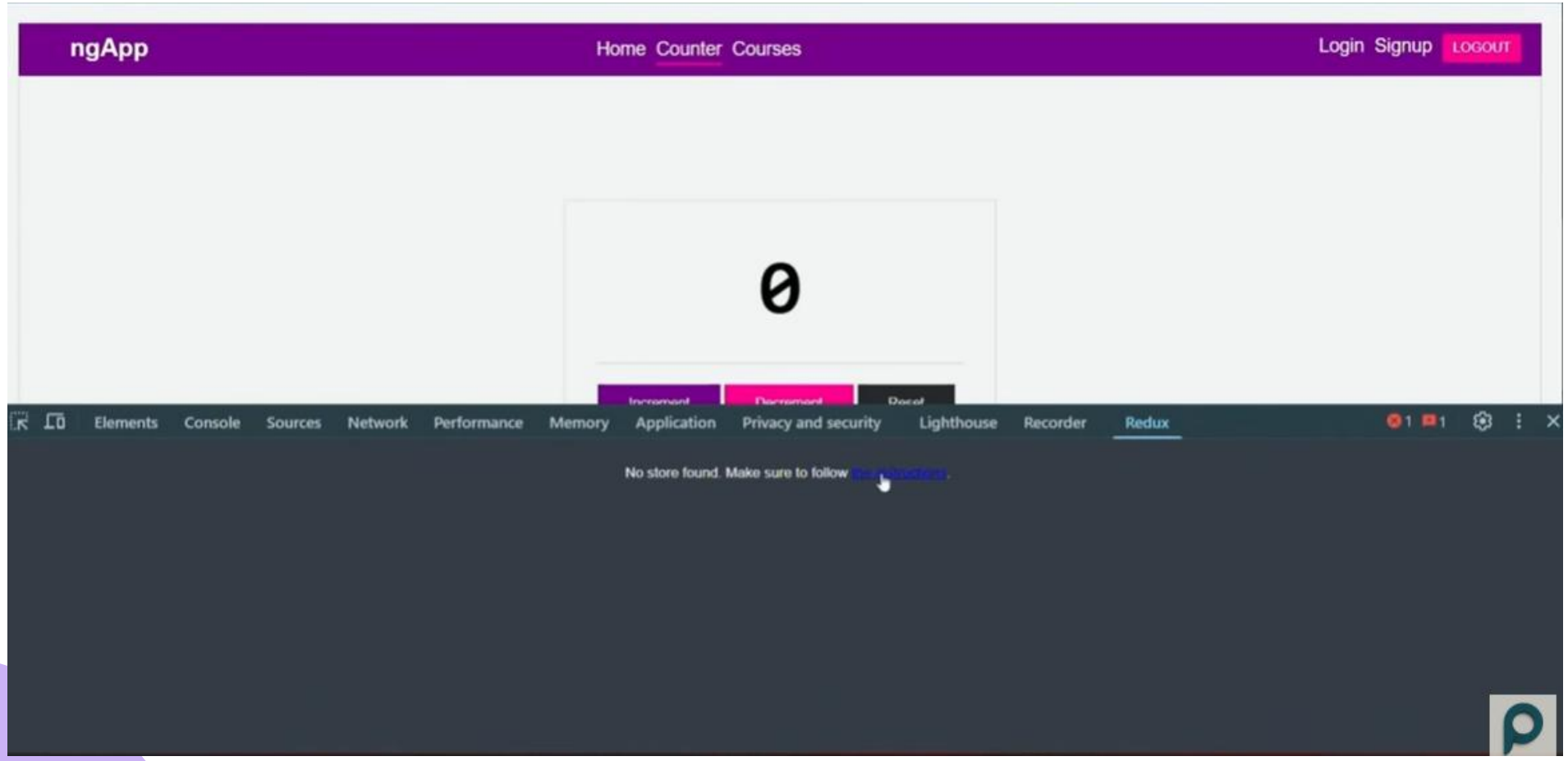
- Inspect app state
- Track dispatched actions
- Time-travel debug
- Replay state changes

Installation steps

First, add the Redux Chrome extension



Assuming you already have a project with NgRx setup, you can inspect the page and go to the Redux tab (provided by the chrome extension)



At this point, you'll notice that it shows "No store found". That's because we didn't yet set it up in the project.

Next, you should run the following command in your terminal to install devtools to your project.

```
ng add @ngrx/store-devtools@latest
```

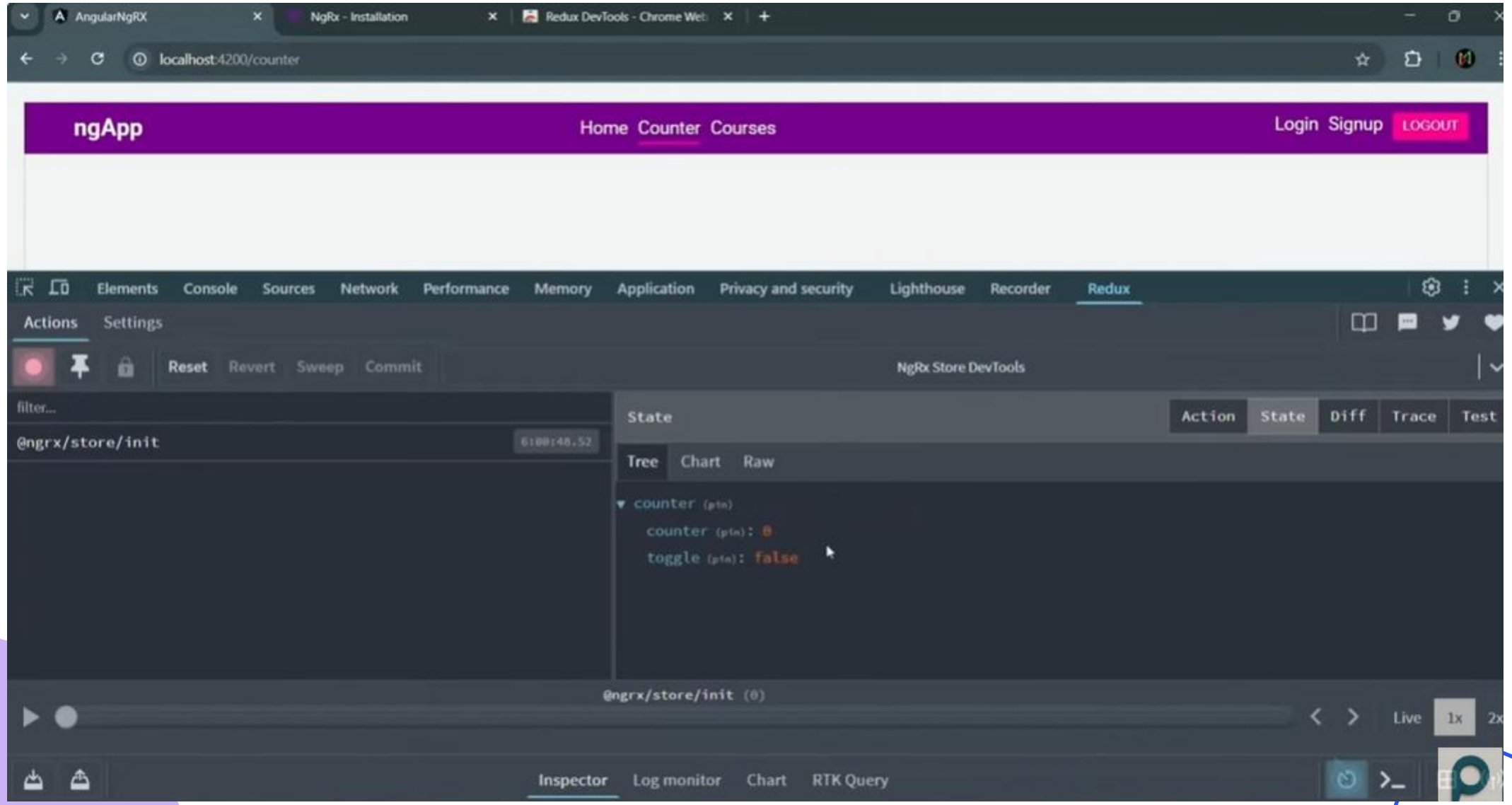
app.config.ts

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideStoreDevtools({
      maxAge: 25,
      logOnly: false,
      autoPause: true,
      features: {
        pause: false,
        lock: true,
        persist: true,
      },
    }),
  ],
};
```

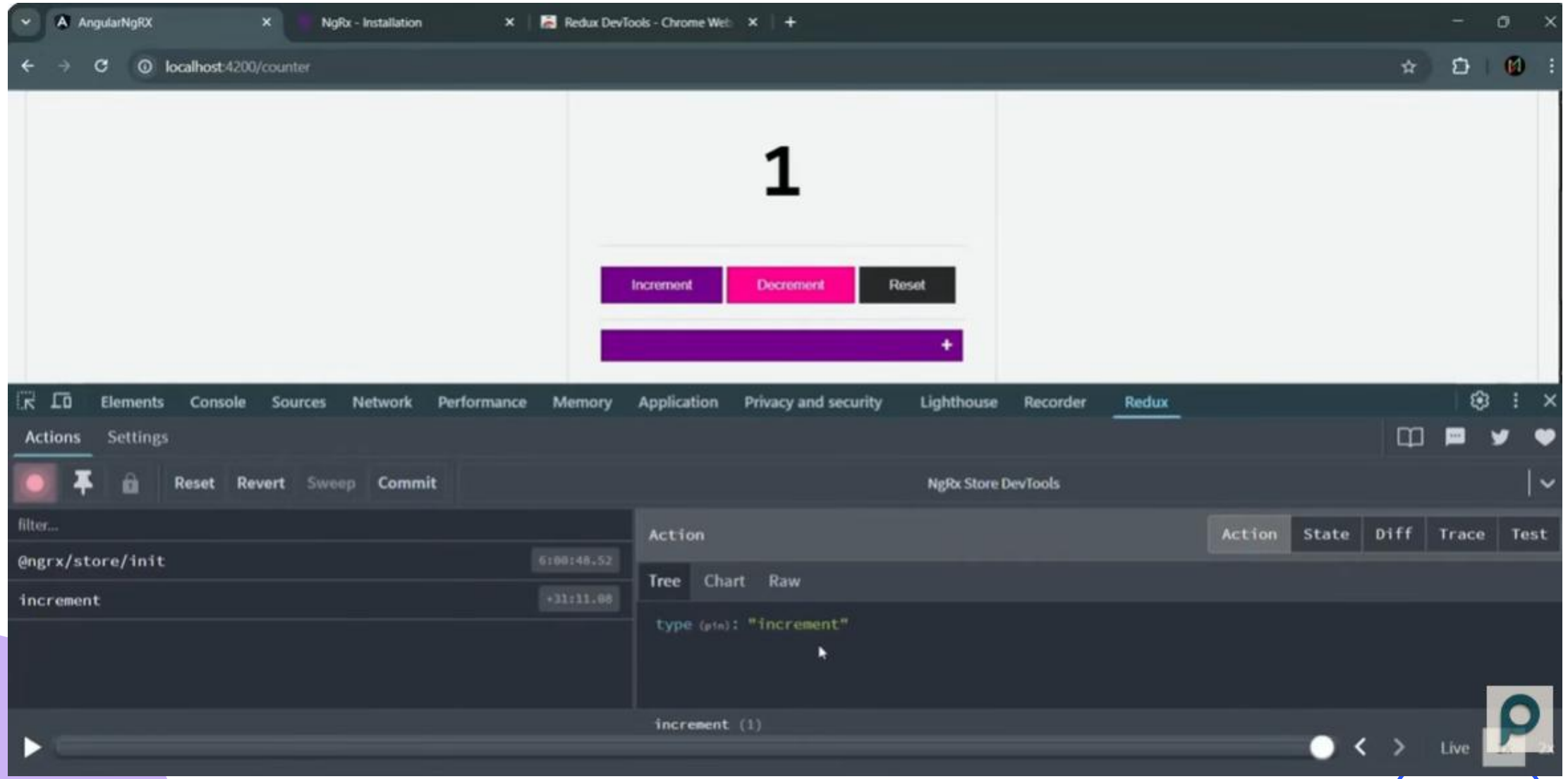
If config isn't automatically set up with devTools like so, you should make sure to add it to your providers.

It comes with a bunch of options that you can add to your config, but they're not necessary. Learn more about the options through the docs.

Now that we set it up, we should be able to see the state in the Redux tab.



If we click on the “Increment” button, we can see the action dispatched and the state afterwards.



Check out this YouTube video for a full demo:

https://www.youtube.com/watch?v=2S1Bs-HwTYo

Signal store and state

inmind.academy
by inmind.ai

A signalStore is a lightweight, reactive state container that uses Angular signals instead of RxJS observables. You define it as a function-based construct

```
export const BooksStore = signalStore(
  withState({ books: [], loading: false }),
  withComputed(({ books }) => ({
    bookCount: computed(() => books().length),
  })),
  withMethods(({ books, ...store }) => ({
    addBook(book: Book) {
      patchState(store, { books: [...books(), book] });
    },
  }))
);
```

signalState is a simpler, lower-level primitive. It creates a reactive signal-based state object that you can use inside a service or component without the full signalStore machinery. Think of it as "just the state part" without the opinionated structure.

```
const state = signalState({ count: 0, name: 'Alice' });
// Access: state.count() — each property becomes a signal
// Update: patchState(state, { count: 1 });
```

What advantages does it offer compared to regular signals?

Signals in a service approach:

```
@Injectable({ providedIn: 'root' })
export class BooksService {
  private _books = signal<Book[]>([]);
  private _loading = signal(false);

  books = this._books.asReadonly();
  loading = this._loading.asReadonly();
  bookCount = computed(() => this._books().length);

  addBook(book: Book) {
    this._books.update(prev => [...prev, book]);
  }
}
```

Instead of managing multiple individual signals and updating them one by one, patchState lets you update multiple properties in a single call:

```
patchState(store, { books: newBooks, loading: false, error: null });
```

(There's also something called entity adapters that help you automate operations on different collections that you can look into)

Performance Optimization

Unsubscribing on Destroy

Unsubscribing on destroy is important because subscriptions keep running even after a component is removed.

Why that matters:

- **Prevents memory leaks** → Active subscriptions hold references to the destroyed component, so it can't be garbage-collected.

- **Avoids unwanted side effects** → Streams may still emit values and try to update a view that no longer exists.

- **Prevents duplicate work** → Re-entering the component can create multiple subscriptions to the same stream.

Rule of thumb:
If an Observable doesn't complete on its own, unsubscribe when the component is destroyed (or use async pipe or signals).

Lazy loading

Lazy loading means loading a feature only when it’s needed, instead of bundling it in the initial app load.

Why it’s useful?

- Smaller initial bundle
- Faster app startup
- Better performance for large apps

Example: Admin pages load only when the user navigates to /admin.

```ts
// app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: '',
    redirectTo: 'dashboard',
    pathMatch: 'full'
  },

  // Lazy load a SINGLE standalone component
  {
    path: 'dashboard',
    loadComponent: () =>
      import('./dashboard/dashboard.component')
        .then(m => m.DashboardComponent)
  },

  // Lazy load a FEATURE (multiple routes)
  {
    path: 'admin',
    loadChildren: () =>
      import('./admin/admin.routes')
        .then(m => m.ADMIN_ROUTES)
  }
];
```

```ts
import { Routes } from '@angular/router';
import { AdminComponent } from './admin.component';
import { UsersComponent } from './users.component';
import { SettingsComponent } from './settings.component';

export const ADMIN_ROUTES: Routes = [
  {
    path: '',
    component: AdminComponent,
    children: [
      {
        path: 'users',
        component: UsersComponent
      },
      {
        path: 'settings',
        component: SettingsComponent
      },
      {
        path: '',
        redirectTo: 'users',
        pathMatch: 'full'
      }
    ]
  }
];
```

Use loadComponent when:

- One page
- No child routes
- Lightweight feature

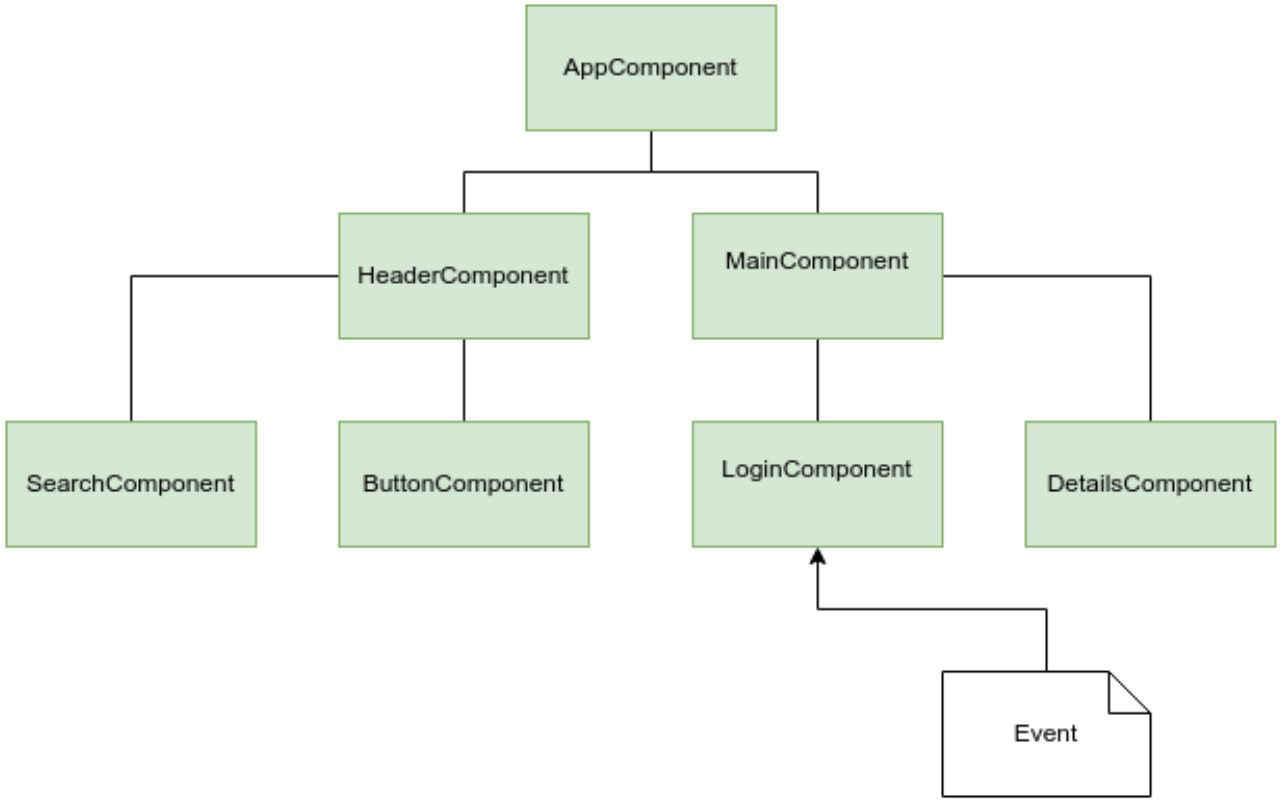
Use loadChildren when:

- Multiple pages related to a feature
- Separate feature with separate routes for organization

OnPush

Change detection reminder

Whenever any event occurs anywhere in your application (clicks, HTTP responses, timers, etc.), Angular checks every component in the entire component tree to see if any data has changed that would require updating the DOM.



OnPush CD

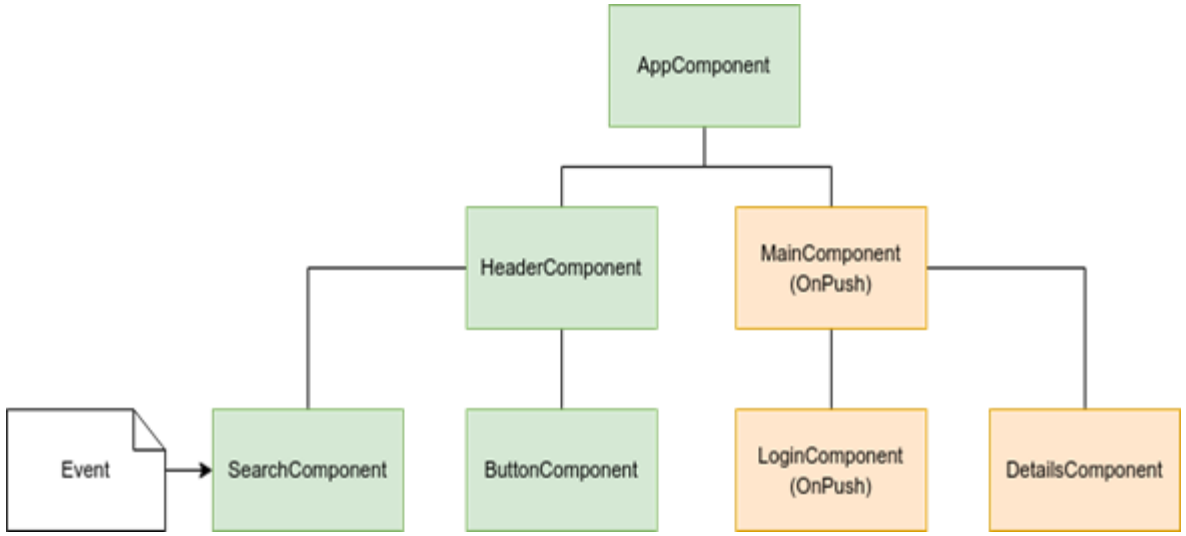
inmind.academy
by inmind.ai

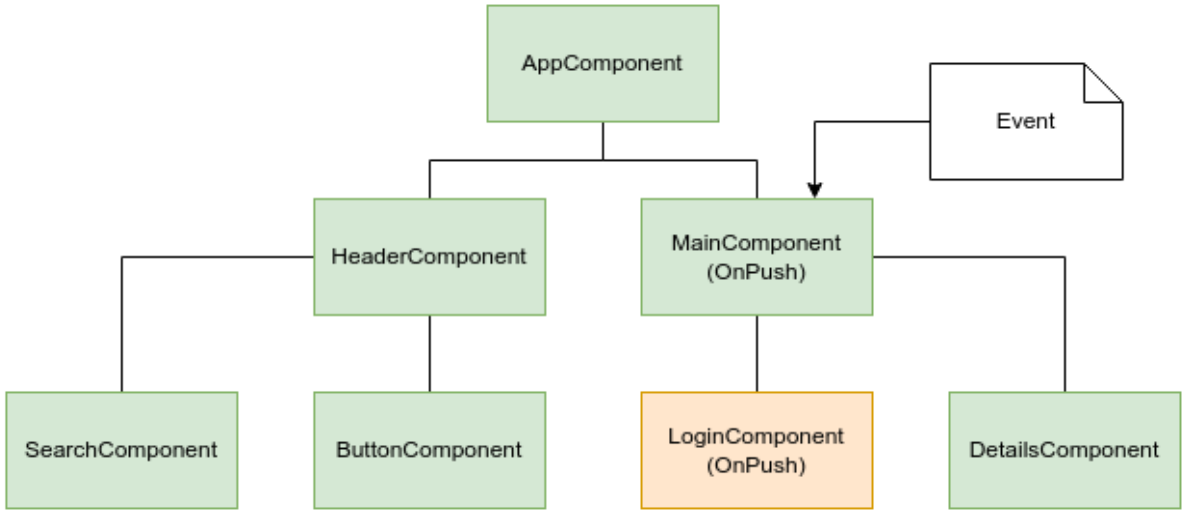
OnPush is an optimization strategy in Angular that changes when and how the framework checks a component for updates.

Specifically, Angular will only run change detection on an OnPush component when:

- An input property **reference** of that component changes - A new object/array is passed to an @Input(), not just a mutation of the existing one
- An event originates from the component or its children - Like a click handler within that component's template
- An observable linked to the async pipe emits - The component uses | async and the observable emits a new value
- Change detection is manually triggered - Using ChangeDetectorRef.markForCheck() or detectChanges()
- A signal updates

Benefit: Skip unnecessary checks for components whose data hasn't actually changed


AppComponent
HeaderComponent
MainComponent
(OnPush)
Event
SearchComponent
ButtonComponent
LoginComponent
(OnPush)
DetailsComponent


AppComponent
Event
HeaderComponent
MainComponent
(OnPush)
SearchComponent
ButtonComponent
LoginComponent
(OnPush)
DetailsComponent

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <p>Count: {{ count() }}</p>
    <p>Double: {{ doubled() }}</p>
    <button (click)="increment()">+1</button>
  `
})
export class CounterComponent {
  count = signal(0);
  doubled = computed(() => this.count() * 2);

  increment() {
    this.count.update(c => c + 1);
  }
}
```

inmind.academy
by inmind.ai

THANK YOU

Questions?

ADDRESS

Mkalles, main road, bld. 757, Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA